SafeImpute

Plateforme sécurisée d'imputation de valeurs de laboratoire clinique

Spécification technique · Architecture produit · Référence d'ingénierie

Type de document	Document d'exigences produit · Architecture de solution · Spécification de conception technique
Pile technologique	Frontend : React 18 · TypeScript · Vite · Tailwind CSS
	Backend : Python · FastAPI · PyTorch · Opacus
Réalisé par	Rahmani Mohammed
Étudiants	Ismail · Dialo · Felix
Encadrants	Rachid Naciri · Mohamed Amine Madani

Table des matières

1. Résumé exécutif **3**

2. Problème clinique **4**

2.1 Limites des méthodes d'imputation classiques 5

3. Objectifs du projet **5**

4. Vue d'ensemble de la solution **5**

5. Fonctionnalités du produit **6**

5.1 Authentification & gestion de session 6

5.2 Gestion des panels patients 6

5.3 Espace de travail d'imputation 6

5.4 Tableau de bord d'évaluation 6

5.5 Rapport de confidentialité 7

5.6 Administration 7

6. Rôles utilisateurs & contrôle d'accès **7**

6.1 Définitions des rôles 7

6.2 Matrice des permissions 7

6.3 Comportement de l'interface en fonction du rôle 8

6.4 Implémentation technique 8

7. Architecture du système **8**

7.1 Protocole de communication 9

7.2 Responsabilités des niveaux 10

7.3 Modèle de stockage 10

8. Architecture du frontend **10**

8.1 Écrans de l'application 11

8.2 Composants principaux 11

8.3 Structure du dépôt 12

9. Architecture du backend **12**

9.1 Endpoints de l'API 12

9.2 Authentification & autorisation 13

9.3 Structure du dépôt 13

10. Moteur d'apprentissage automatique **13**

10.1 Architecture du Masked Transformer 13

10.2 Modèles de référence 14

10.3 Chemin d'entraînement à confidentialité différentielle 14

11. Confidentialité & sécurité **15**

11.1 Modèle de menace 15
11.2 Confidentialité différentielle (DP-SGD) 15
11.3 Audit d'inférence d'appartenance 15
11.4 Test de distance aux plus proches voisins 15
11.5 Sécurité applicative 16
12. Cadre d'évaluation 16
13. Flux de travail de bout en bout 17
13.1 Imputation d'un panel unique (Clinicien) 17
13.2 Imputation par lots (Clinicien / Admin) 17
13.3 Flux de travail d'évaluation (Chercheur) 17
13.4 Flux de travail d'audit de confidentialité (Chercheur / Admin) 17
13.5 Flux de travail d'entraînement du modèle (Admin) 18
14. Architecture de déploiement 18
14.1 Développement local 18
14.2 Déploiement en production 18
15. Pile technologique 19
16. Sources de données 20
16.1 Interface DatasetLoader 20
17. Extensions futures 21

1. Résumé exécutif

SafeImpute est une plateforme full-stack préservant la confidentialité, destinée à l'imputation des valeurs manquantes dans les panels de laboratoire clinique. Elle combine un réseau de neurones Masked Transformer léger avec un cadre d'entraînement de confidentialité différentielle et une suite d'évaluation complète, livrée sous forme d'application web de niveau production avec des composants frontend et backend clairement séparés.

Plateforme :	SafeImpute, imputation sécurisée de valeurs de laboratoire clinique
Frontend :	React 18 · TypeScript · Vite · Tailwind CSS
Backend :	Python · FastAPI · PyTorch · Opacus
Modèle :	Masked Transformer (~200k paramètres, entraînement < 1 h sur un seul GPU)
Confidentialité :	Confidentialité différentielle (DP-SGD via Opacus) + audit MIA + test NN-Distance
Accès :	Contrôle d'accès basé sur les rôles, Admin · Clinicien · Chercheur

La plateforme répond à un défi répandu en informatique clinique : les panels de laboratoire incomplets créent des lacunes qui dégradent les modèles de prédiction en aval. SafeImpute comble ces lacunes avec des valeurs imputées statistiquement fondées et conscientes de l'incertitude, tout en fournissant des garanties formelles de confidentialité qui protègent les données des patients.

Capacités clés :

- Imputation générative des analytes manquants via un Masked Transformer (μ et σ² par valeur imputée)
- Quantification de l'incertitude, chaque valeur imputée est livrée avec un intervalle de confiance calibré
- Chemin d'entraînement à confidentialité différentielle (DP-SGD) avec budget de confidentialité ε configurable
- Suite d'audit de confidentialité : test Membership Inference Attack (MIA) et analyse de distance aux plus proches voisins
- Évaluation à quatre axes : précision de reconstruction · fidélité distributionnelle · impact clinique en aval · calibration de l'incertitude
- Contrôle d'accès basé sur les rôles appliqué indépendamment au niveau de l'API et de l'interface (défense en profondeur)
- Superposition de comparaison de référence : Masked Transformer vs. moyenne · k-NN · MICE

2. Problème clinique

Les panels de laboratoire clinique contiennent des valeurs manquantes fréquentes et structurellement inévitables. Trois mécanismes provoquent cette absence de données :

- Prescription sélective des tests, les cliniciens ne demandent que les analytes pertinents pour la question clinique en cours
- Arrivée asynchrone des résultats, les tests au point de soin reviennent en quelques minutes ; les cultures prennent plus de 48 heures
- Historiques de patients fragmentés, les panels collectés lors de différentes visites diffèrent dans leur composition d'analytes

Le résultat est une matrice de données présentant des lacunes systématiques. Le tableau ci-dessous illustre un dossier patient représentatif (les cellules en rouge indiquent des valeurs manquantes) :

Temps	CRP	Leucocytes	Neutrophiles	Lactate	Plaquettes	Créatinine
h1	12	9.1	—	—	210	0.9
h2	—	9.4	7.2	1.8	—	—
h3	18	—	—	—	198	1.0

Les modèles modernes de prédiction clinique, par exemple les systèmes d'alerte précoce du sepsis, exigent des vecteurs de caractéristiques complets. Les lignes comportant des valeurs manquantes sont soit écartées (réduisant le jeu de données effectif), soit remplies naïvement, ce qui dégrade la calibration du modèle. SafeImpute résout ce problème en fournissant des complétions imputées principiées et conscientes de l'incertitude.

2.1 Limites des méthodes d'imputation classiques

Méthode	Mode de défaillance	Pourquoi SafeImpute la surpasse
Moyenne / Médiane	Détruit les corrélations inter-analytes	Le Masked Transformer capture les distributions conjointes d'analytes via l'auto-attention
k plus proches voisins	Coût de requête O(N·D) ; sensible à k et à la métrique de distance	Inférence en une seule passe avant ; aucune recherche de plus proches voisins à l'exécution
MICE	Suppose des relations linéaires entre les analytes	L'auto-attention capture les relations biologiques non linéaires et multimodales

3. Objectifs du projet

- Livrer une application web full-stack de qualité production pour l'imputation des panels de laboratoire clinique
- Implémenter un modèle Masked Transformer entraîné sur les jeux de données PhysioNet 2019 et MIMIC-IV
- Fournir une quantification de l'incertitude (μ, σ²) pour chaque valeur imputée
- Intégrer un chemin d'entraînement à confidentialité différentielle (DP-SGD via Opacus)
- Implémenter un cadre d'évaluation complet à quatre axes (reconstruction · fidélité · clinique · calibration)
- Appliquer le contrôle d'accès basé sur les rôles sur toutes les surfaces de la plateforme
- Livrer un module d'audit de confidentialité (test MIA + analyse de distance aux plus proches voisins)
- Concevoir la couche de données derrière une interface DatasetLoader interchangeable pour soutenir une future intégration hospitalière

4. Vue d'ensemble de la solution

SafeImpute est une application découplée à deux niveaux. Un frontend React TypeScript communique avec un backend Python FastAPI via REST/JSON. Toute l'inférence d'apprentissage automatique s'exécute côté serveur ; le navigateur n'exécute jamais de code de modèle et n'accède jamais aux données brutes d'entraînement.

Le composant ML central est un Masked Transformer adapté aux données tabulaires numériques. Le modèle apprend la distribution conjointe des analytes de laboratoire en s'entraînant sur des panels masqués aléatoirement, puis utilise cette distribution pour reconstruire les valeurs manquantes lors de l'inférence. Le résultat est une distribution prédictive (μ, σ²), et non une estimation ponctuelle, ce qui permet une aide à la décision clinique consciente de l'incertitude.

Modèle d'imputation :	Masked Transformer · ~200k paramètres · < 1 h d'entraînement sur un seul GPU
Couche de confidentialité :	DP-SGD (Opacus) · budget ε configurable · audit MIA · test NN-distance
Jeux de données :	PhysioNet 2019 Sepsis Challenge · MIMIC-IV Demo · Générateur synthétique de repli
API :	FastAPI REST · authentification JWT · middleware RBAC · documentation OpenAPI
Frontend :	React 18 · TypeScript · Tailwind CSS · Recharts

5. Fonctionnalités du produit

5.1 Authentification & gestion de session

- Connexion basée sur JWT avec hachage des mots de passe par bcrypt
- Jeton de session transporté dans l'en-tête Authorization à chaque appel API
- Rôle résolu à la connexion et intégré dans les revendications du JWT
- Les routes protégées redirigent les utilisateurs non autorisés vers un écran 403

5.2 Gestion des panels patients

- Saisie manuelle des analytes via un formulaire de panel structuré
- Téléversement CSV pour l'imputation par lots de plusieurs dossiers patients
- Masque d'absence de données automatiquement déduit des champs vides
- Historique des panels par patient, consultation, ré-imputation, comparaison dans le temps

5.3 Espace de travail d'imputation

- Imputation d'un seul panel : saisir les valeurs observées → recevoir les valeurs imputées + intervalles de confiance
- Imputation par lots : téléverser un CSV → télécharger les panels complétés
- Indicateurs visuels de confiance : bande étroite = confiance élevée, bande large = confiance faible
- Superposition de comparaison de méthodes : Masked Transformer vs. moyenne / k-NN / MICE affichés côte à côte
- Mode d'imputation multiple : K échantillons tirés de la distribution prédictive pour la propagation de l'incertitude

5.4 Tableau de bord d'évaluation

- Précision de reconstruction : RMSE et MAE par analyte
- Fidélité distributionnelle : distance de Wasserstein et Maximum Mean Discrepancy (MMD)
- Impact clinique en aval : AUC et ECE sur la tâche de prédiction du sepsis
- Calibration de l'incertitude : couverture empirique vs. niveau de confiance déclaré

- Panneaux Recharts interactifs pour les quatre axes d'évaluation

5.5 Rapport de confidentialité

- Score AUC du MIA, cible ≈ 0.5 (indiscernable d'un classificateur aléatoire)
- Distribution de distance aux plus proches voisins : complétions imputées vs. proximité réelle des patients
- Budget de confidentialité différentielle ε actif et mode d'entraînement (DP-SGD activé / désactivé)
- Rapport d'attestation de confidentialité exportable (téléchargement JSON + vue dans l'interface)

5.6 Administration

- Gestion des utilisateurs : créer, modifier et désactiver des comptes
- Attribution des rôles : Admin / Clinicien / Chercheur
- Configuration des jeux de données : enregistrer de nouvelles sources de données
- Lancement de l'entraînement du modèle : déclencher des sessions d'entraînement depuis l'interface (Admin uniquement)
- Journal d'activité : piste d'audit complète des actions des utilisateurs avec horodatage

6. Rôles utilisateurs & contrôle d'accès

6.1 Définitions des rôles

Rôle	Cas d'usage principal	Permissions clés
Admin	Administrateur de la plateforme	Accès complet à toutes les ressources, gestion des utilisateurs, configuration système, orchestration de l'entraînement
Clinicien	Utilisateur clinique réalisant l'imputation	Saisir des panels, exécuter l'imputation, consulter les résultats et les bandes d'incertitude, exporter des résultats individuels
Chercheur	Chercheur évaluant la qualité du modèle	Exécuter des évaluations, consulter le tableau de bord d'évaluation et les rapports de confidentialité, lancer des expériences d'entraînement/ DP-SGD

6.2 Matrice des permissions

La matrice ci-dessous définit les permissions pour chaque combinaison (ressource × action × rôle).

Ressource / Action	Voir			Modifier			Exécuter			Exporter			Gérer		
	A	C	R	A	C	R	A	C	R	A	C	R	A	C	R
Patients / Panels	✓	✓	✓	✓	✓	–	✓	✓	–	✓	✓	–	✓	–	–
Imputation	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	–	✓	–	–
Évaluation	✓	–	✓	✓	–	✓	✓	–	✓	✓	–	✓	✓	–	–
Rapports de confidentialité	✓	–	✓	✓	–	–	–	–	–	✓	–	✓	✓	–	–
Jeux de données	✓	–	✓	✓	–	✓	–	–	–	✓	–	–	✓	–	–
Utilisateurs	✓	–	–	✓	–	–	–	–	–	–	–	–	✓	–	–
Journaux d'activité	✓	–	–	–	–	–	–	–	–	–	–	–	–	–	–

Légende des rôles : A = Admin · C = Clinicien · R = Chercheur ✓ = Autorisé – = Non autorisé

6.3 Comportement de l'interface en fonction du rôle

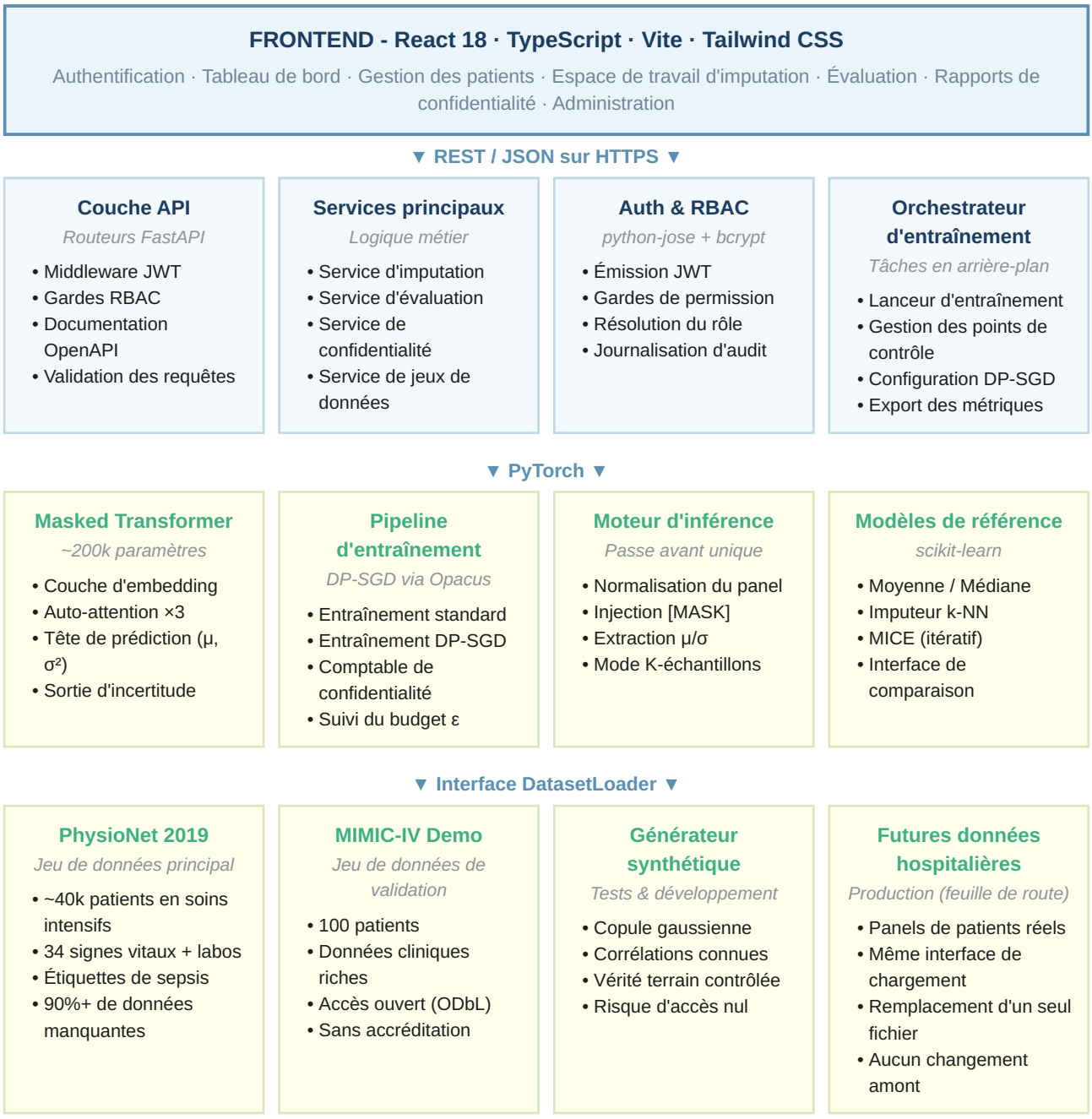
- Les éléments de la barre de navigation latérale s'affichent de manière conditionnelle selon le rôle de l'utilisateur authentifié
- Les boutons d'action (Ré-entraîner, Exporter, Gérer les utilisateurs) sont masqués pour les rôles ne disposant pas de la permission correspondante
- Les routes sont protégées côté client (composant RequirePermission) et côté serveur (dépendance FastAPI), défense en profondeur
- Un Clinicien voit une interface simplifiée de saisie de panel et d'imputation
- Un Chercheur voit les surfaces d'évaluation et de confidentialité
- Un Admin voit la console de gestion complète, incluant la gestion des utilisateurs, la configuration des jeux de données et les journaux d'activité

6.4 Implémentation technique

- Backend : table User (id, email, bcrypt_hash, role) dans SQLite ; les revendications JWT transportent le rôle
- Chaque endpoint FastAPI protégé déclare Depends(require(resource, action))
- La dépendance lit le JWT, résout la carte des permissions du rôle, et lève une erreur HTTP 403 en cas d'incohérence
- Frontend : AuthContext contient l'utilisateur + les permissions ; le HOC RequirePermission encapsule les routes et les composants
- Un script d'amorçage (seed) crée un compte par rôle pour un accès de démonstration immédiat

7. Architecture du système

SafeImpute est structuré en un système à quatre niveaux : frontend React basé sur le navigateur → backend Python FastAPI → moteur ML PyTorch → couche de données interchangeable. Chaque niveau a des responsabilités clairement délimitées, sans fuite entre niveaux.



7.1 Protocole de communication

- Toute communication client-serveur utilise REST sur HTTPS avec des charges utiles JSON en requête/réponse
- Le frontend attache un Bearer JWT à chaque requête authentifiée via un intercepteur Axios
- Les calculs lourds (entraînement, évaluation par lots, audits de confidentialité) sont découplés du cycle de vie de la requête HTTP ; les résultats sont écrits dans des artefacts JSON par des scripts hors ligne et servis via des endpoints de lecture mis en cache

7.2 Responsabilités des niveaux

Frontend (React)	Affiche l'interface, gère la saisie utilisateur, appelle les endpoints REST, affiche les résultats et les graphiques. Aucun code ML, aucun accès aux données brutes.
Backend (FastAPI)	Applique l'authentification et le RBAC, valide les requêtes, délègue aux modules ML/évaluation/ confidentialité, lit/écrit les artefacts de stockage.
Moteur ML (PyTorch)	Entraîne les modèles, exécute l'inférence, calcule les métriques d'évaluation, exécute les audits de confidentialité. Fonctionne en Python pur, appelable depuis la CLI ou via FastAPI.
Couche de données	Fournit une paire DataFrame standardisée (X, mask) via l'interface DatasetLoader. Le remplacement des jeux de données ne nécessite qu'une modification d'un seul fichier.

7.3 Modèle de stockage

Poids du modèle	Fichiers .pt sur le système de fichiers du backend, chargés au démarrage du serveur ; remplaçables à chaud
Résultats d'évaluation	metrics.json, écrit par evaluate.py hors ligne ; servi depuis l'endpoint /metrics
Résultats d'audit de confidentialité	privacy_report.json, écrit par run_privacy_audit.py ; servi depuis /privacy/report
Comptes utilisateurs & rôles	Table SQLite users (id, email, bcrypt_hash, role)
Journaux d'activité	Table SQLite audit_log (user_id, action, resource, timestamp)
Fichiers de jeux de données	Système de fichiers, format PSV/CSV ; accessibles via l'interface DatasetLoader

8. Architecture du frontend

Le frontend est une application React monopage construite avec Vite. Elle est totalement découplée de la couche ML ; aucun code de modèle ne s'exécute dans le navigateur, et aucune donnée brute d'entraînement n'est jamais transmise au client.

8.1 Écrans de l'application

Écran	Route	Accès par rôle
Connexion	/login	Public
Tableau de bord	/dashboard	Tous les rôles
Éditeur de panel	/panels/new	Admin · Clinicien
Gestion des patients	/patients	Admin · Clinicien
Vue d'imputation	/impute/:id	Admin · Clinicien · Chercheur
Comparaison de méthodes	/compare	Admin · Chercheur
Tableau de bord d'évaluation	/evaluation	Admin · Chercheur
Rapport de confidentialité	/privacy	Admin · Chercheur
Gestion des utilisateurs	/admin/users	Admin uniquement
Configuration des jeux de données	/admin/datasets	Admin uniquement
Journal d'activité	/admin/logs	Admin uniquement

8.2 Composants principaux

- AuthContext : contient l'utilisateur authentifié, le rôle et la carte des permissions ; peuplé à la connexion, vidé à la déconnexion
- RequirePermission : HOC/wrapper qui vérifie (ressource, action) avant d'afficher une route ou un composant
- Layout : barre latérale consciente des rôles qui affiche conditionnellement les éléments de navigation selon les permissions
- Couche API : client Axios avec intercepteur JWT ; hooks TanStack Query pour la récupération des données et l'invalidation du cache
- PanelEditor : formulaire structuré pour la saisie des analytes avec support de téléversement CSV et déduction automatique du masque
- ImputationView : affiche les valeurs imputées avec visualisation des bandes d'incertitude Recharts
- EvalDashboard : mise en page à quatre panneaux (un graphique par axe d'évaluation)
- PrivacyReport : jauge AUC du MIA, histogramme NN-distance, affichage ε, bouton d'export JSON

8.3 Structure du dépôt

```
frontend/
├─ src/
│   ├─ pages/          # Login, Dashboard, PanelEditor, ImputationView,
│   │                    #   Evaluation, Privacy, Compare, Users (admin)
│   ├─ auth/           # AuthContext.tsx, RequirePermission.tsx
│   ├─ components/     # Composants UI partagés (graphiques, tableaux, badges)
│   ├─ api/            # client axios (intercepteur JWT) + hooks TanStack Query
│   └─ types/          # Interfaces et définitions de types TypeScript
└─ tests/              # Tests de composants et d'intégration Vitest
```

9. Architecture du backend

Le backend est une application Python FastAPI. Il expose une API REST, applique l'authentification et le RBAC, et délègue les tâches d'imputation, d'évaluation et d'audit de confidentialité aux modules ML et d'évaluation. Les modules ML sont indépendants de FastAPI et peuvent être appelés directement depuis la ligne de commande pour des expériences.

9.1 Endpoints de l'API

Méthode	Endpoint	Accès	Description
POST	/auth/login	Public	Authentifie les identifiants ; retourne un JWT signé
GET	/auth/me	Authentifié	Retourne le profil de l'utilisateur courant et les permissions résolues
GET	/users	Admin	Liste tous les utilisateurs de la plateforme
POST	/users	Admin	Crée un nouveau compte utilisateur avec un rôle
PUT	/users/:id	Admin	Met à jour le rôle ou désactive un compte
POST	/impute	Clinicien+	Impute les valeurs manquantes dans un panel unique ; retourne μ et σ
POST	/impute/batch	Clinicien+	Imputation par lots à partir d'un CSV téléversé ; retourne les panels complétés
GET	/metrics	Chercheur+	Retourne les résultats d'évaluation mis en cache depuis metrics.json
GET	/privacy/report	Chercheur+	Retourne les résultats d'audit de confidentialité mis en cache depuis privacy_report.json
POST	/train	Admin	Déclenche une session d'entraînement du modèle (tâche en arrière-plan)
GET	/datasets	Chercheur+	Liste les sources de jeux de données enregistrées
GET	/audit/logs	Admin	Retourne les entrées paginées du journal d'activité

9.2 Authentification & autorisation

- JWT émis à /auth/login, signé avec HMAC-SHA256, transportant { user_id, email, role }
- Chaque endpoint protégé déclare Depends(require(resource, action)), injection de dépendances FastAPI
- Le garde résout le rôle depuis le JWT, vérifie la carte des permissions, et lève une erreur HTTP 403 si l'action n'est pas autorisée
- Les mots de passe sont stockés sous forme de hachages bcrypt, le texte en clair n'est jamais persisté ni journalisé

9.3 Structure du dépôt

```
backend/
├─ app/
│   ├─ main.py             # Point d'entrée FastAPI + enregistrement des routeurs
│   ├─ routers/            # auth.py, users.py, impute.py, metrics.py,
│   │                       #   privacy.py, train.py, datasets.py, audit.py
│   ├─ auth/               # jwt.py, password.py, roles.py, guards.py
│   ├─ ml/
│   │   ├─ model.py        # Architecture MaskedTransformer
│   │   ├─ trainer.py      # Boucle d'entraînement standard + DP-SGD
│   │   ├─ imputer.py      # Inférence + incertitude K-échantillons
│   │   └─ baselines.py    # Wrappers Moyenne / k-NN / MICE
│   ├─ eval/               # reconstruction.py, fidelity.py,
│   │                       #   downstream.py, calibration.py
│   ├─ privacy/            # mia.py, nn_distance.py
│   └─ data/               # loaders.py, preprocessing.py, synthetic.py
├─ scripts/
│   ├─ train.py            # Lanceur d'entraînement hors ligne
│   ├─ evaluate.py         # Génère metrics.json
│   └─ run_privacy_audit.py # Génère privacy_report.json
└─ tests/                  # Tests unitaires et d'intégration pytest
```

10. Moteur d'apprentissage automatique

10.1 Architecture du Masked Transformer

Le modèle central est un Masked Transformer de style BERT adapté aux données tabulaires numériques. Il apprend la distribution conjointe des analytes de laboratoire en s'entraînant sur des panels masqués aléatoirement, et lors de l'inférence reconstruit les valeurs manquantes à partir du contexte observé.

#	Étape	Détail
1	**Tokénisation & Embedding**	Chaque analyte est associé à un vecteur dense (dim 64–128) via une couche d'embedding apprise. Des embeddings positionnels de type analyte indiquent quelle mesure représente chaque jeton.
2	**Masquage aléatoire (entraînement)**	15–40% des analytes par panel sont remplacés par un jeton [MASK]. Le modèle apprend à retrouver la vraie valeur à partir du contexte observé restant.
3	**Encodage par auto-attention**	2–4 couches d'encodeur Transformer. Chaque analyte porte attention à tous les autres, capturant les corrélations non linéaires inter-analytes (par ex. co-élévation CRP ↔ Leucocytes).
4	**Tête de prédiction**	Pour chaque position masquée, une tête linéaire produit (μ, σ^2), une moyenne et une variance prédictives, permettant une quantification de l'incertitude par imputation.
5	**Fonction de perte (entraînement)**	Log-vraisemblance négative sur la gaussienne prédictive. Le modèle minimise l'erreur de prédiction tout en apprenant à calibrer sa propre confiance.
6	**Inférence**	Les analytes manquants sont masqués. Une passe avant retourne μ (valeur imputée) et σ (incertitude). K échantillons en passe avant génèrent la bande de confiance complète.

Taille du modèle : ~200 000 paramètres (dimension d'embedding 128, 3 couches d'attention)

Temps d'entraînement : < 1 heure sur un seul GPU (réalisable sur le palier gratuit T4 de Google Colab)

Inférence : Passe avant unique, pas d'échantillonnage itératif ; pas de recherche de plus proches voisins

10.2 Modèles de référence

- Imputation par moyenne / médiane, remplacement par moyenne par analyte (scikit-learn SimpleImputer)
- k plus proches voisins, KNNImputer de scikit-learn
- MICE, IterativeImputer (imputation multiple par équations chaînées) de scikit-learn
- Tous les modèles de référence partagent la même interface Imputer que le Transformer pour une comparaison équitable et automatisée

10.3 Chemin d'entraînement à confidentialité différentielle

- DP-SGD implémenté via Opacus (Meta), un wrapper d'entraînement PyTorch « drop-in »
- L'écrêtage de gradient par échantillon limite l'influence de tout patient individuel sur les poids du modèle
- Un bruit gaussien calibré est ajouté aux gradients écrêtés avant chaque mise à jour de l'optimiseur
- Un comptable de confidentialité suit la dépense cumulée (ε, δ) au cours des étapes d'entraînement
- Deux modes d'entraînement : Standard (sans confidentialité) et DP-SGD (ε configurable)
- La valeur ε active est rapportée sur l'écran Rapport de confidentialité aux côtés des résultats d'audit

11. Confidentialité & sécurité

11.1 Modèle de menace

Vecteur d'attaque	Description	Mitigation SafeImpute
Inférence d'appartenance	Un adversaire détermine si un patient spécifique faisait partie des données d'entraînement	Entraînement DP-SGD + audit MIA visant une AUC ≤ 0.55
Inférence d'attribut	Un adversaire reconstruit des attributs sensibles de patients à partir des sorties du modèle	Bruit de gradient DP-SGD + analyse NN-distance
Extraction de données	Un adversaire récupère des enregistrements d'entraînement quasi complets en sondant le modèle	Test NN-distance post-entraînement ; résultats affichés dans le Rapport de confidentialité
Accès non autorisé	Un utilisateur non autorisé accède aux données de patients ou aux résultats d'imputation	JWT + RBAC appliqués à chaque appel API ; les gardes de l'interface sont complémentaires

11.2 Confidentialité différentielle (DP-SGD)

- Algorithme : DP-SGD (Abadi et al., 2016), écrêtage de gradient par échantillon + bruit gaussien calibré
- Bibliothèque : Opacus (Meta), intégration en une ligne avec les boucles d'entraînement PyTorch existantes
- Le budget de confidentialité ε est configurable au lancement de l'entraînement ; suivi par le comptable de confidentialité d'Opacus
- Une garantie formelle (ε, δ) est rapportée sur l'écran Rapport de confidentialité après l'entraînement

11.3 Audit d'inférence d'appartenance

- Approche par modèles fantômes (shadow models) : des modèles auxiliaires entraînés sur des données retenues simulent un adversaire
- AUC d'attaque mesurée par rapport au modèle de production, cible AUC ≤ 0.55
- Résultats écrits dans privacy_report.json et affichés sur l'écran Rapport de confidentialité

11.4 Test de distance aux plus proches voisins

- Pour chaque complétion générée par le modèle, la distance au plus proche patient réel d'entraînement est calculée
- Si les distances imputé-vers-entraînement correspondent aux distances réel-vers-réel, le modèle n'est pas traçable jusqu'aux individus
- Résultats tracés sous forme d'histogrammes superposés sur l'écran Rapport de confidentialité

11.5 Sécurité applicative

- Mots de passe : bcrypt/argon2, texte en clair jamais stocké ni journalisé
- Jetons JWT : HMAC-SHA256, expiration courte, support de rotation côté serveur
- RBAC : vérifications de permission appliquées au niveau de la dépendance FastAPI, les gardes de l'interface sont complémentaires, non primaires
- Aucune exécution ML dans le navigateur, tout le code de modèle s'exécute côté serveur

12. Cadre d'évaluation

SafeImpute implémente un cadre d'évaluation à quatre axes qui mesure la qualité de l'imputation selon des perspectives orthogonales. Le cadre est agnostique à l'architecture et réutilisable pour la comparaison entre modèles.

Axe	Nom	Métriques	Description
1	**Précision de reconstruction**	RMSE, MAE (par analyte)	Impute des valeurs connues comme présentes ; mesure l'erreur de prédiction ponctuelle par analyte.
2	**Fidélité distributionnelle**	Distance de Wasserstein, MMD	Compare la distribution des valeurs imputées à la distribution réelle dans la population de patients.
3	**Impact clinique en aval**	AUC, ECE (classificateur de sepsis)	Entraîne un modèle de sepsis sur des données brutes avec lacunes vs. complétées par SafeImpute ; mesure l'amélioration de la discrimination et de la calibration.
4	**Calibration de l'incertitude**	Couverture empirique vs. niveau déclaré	Vérifie qu'un intervalle de confiance à 90% déclaré contient la vraie valeur ~90% du temps.

13. Flux de travail de bout en bout

13.1 Imputation d'un panel unique (Clinicien)

Étape	Acteur	Action
1	Clinicien / Navigateur	S'authentifie via POST /auth/login ; le JWT est retourné et stocké en mémoire
2	Clinicien / Navigateur	Ouvre l'Éditeur de panel ; saisit les analytes observés (par ex. CRP = 12, Leucocytes = 9.1) ; laisse vides les analytes manquants
3	Frontend → Backend	POST /impute { observed: { CRP: 12, Leukocytes: 9.1 }, mask: ["Neutrophils", "Lactate", "Creatinine"] }
4	Backend / Imputer	Normalise le panel ; remplace les positions masquées par [MASK] ; exécute la passe avant du MaskedTransformer ; retourne μ et σ par position masquée
5	Backend → Frontend	Réponse JSON : { imputed: { Neutrophils: 5.2, Lactate: 1.4, Creatinine: 0.87 }, uncertainty: { ... } }
6	Frontend	Affiche ImputationView : valeurs imputées avec bandes de confiance colorées (largeur ∝ σ)
7	Clinicien (optionnel)	Clique sur Comparer les méthodes → le frontend appelle l'endpoint des modèles de référence ; un graphique de superposition affiche Transformer vs. moyenne / k-NN / MICE

13.2 Imputation par lots (Clinicien / Admin)

- Le clinicien téléverse un CSV de panels incomplets via l'Éditeur de panel
- Le frontend envoie POST /impute/batch avec le fichier
- Le backend traite chaque ligne via l'Imputer ; retourne le CSV complété
- Le frontend fournit un lien de téléchargement pour les panels complétés

13.3 Flux de travail d'évaluation (Chercheur)

- Le chercheur exécute evaluate.py hors ligne sur une partition retenue des données PhysioNet / MIMIC
- Le script calcule les quatre axes d'évaluation et écrit les résultats dans metrics.json
- Le chercheur ouvre le Tableau de bord d'évaluation, le frontend récupère GET /metrics et affiche les graphiques
- Les résultats sont en lecture seule dans l'interface ; les calculs lourds s'exécutent toujours en script hors ligne

13.4 Flux de travail d'audit de confidentialité (Chercheur / Admin)

- L'Admin ou le Chercheur exécute run_privacy_audit.py, entraîne des modèles fantômes, calcule l'AUC du MIA et les distances NN
- Résultats écrits dans privacy_report.json

- L'écran Rapport de confidentialité récupère GET /privacy/report ; affiche la jauge AUC du MIA et l'histogramme NN-distance
- Si DP-SGD a été utilisé pendant l'entraînement, la valeur ε active est affichée aux côtés des résultats d'audit

13.5 Flux de travail d'entraînement du modèle (Admin)

- L'Admin ouvre le panneau d'Administration et déclenche une session d'entraînement (configurable : époques, ε, jeu de données, mode)
- Le backend lance train.py comme tâche en arrière-plan ; la progression de l'entraînement est journalisée
- À la fin, les nouveaux poids du modèle remplacent le point de contrôle actif ; l'API sert le modèle mis à jour

14. Architecture de déploiement

14.1 Développement local

- Backend : uvicorn app.main:app --reload (port 8000)
- Frontend : npm run dev (port 5173, serveur de développement Vite avec HMR)
- Docker non requis, deux sessions de terminal
- Un script d'amorçage crée un compte Admin, un compte Clinicien et un compte Chercheur pour un accès de démonstration immédiat

14.2 Déploiement en production

Composant	Environnement d'exécution	Notes
Backend FastAPI	uvicorn / gunicorn derrière nginx	Terminaison HTTPS au niveau de nginx ; actifs statiques servis séparément
Frontend React	Build Vite → fichiers statiques	Déployé sur un CDN ou servi via nginx depuis /dist
Poids ML	Système de fichiers / stockage objet	Chargés au démarrage du backend ; remplaçables à chaud sans redémarrage
Base de données SQLite	Système de fichiers local	Migration vers PostgreSQL pour les déploiements multi-instances
Scripts hors ligne	Tâche planifiée / CLI manuelle	train.py, evaluate.py, run_privacy_audit.py, exécutés à la demande

15. Pile technologique

Couche / Composant	Technologie	Justification
Framework Frontend	React 18 + TypeScript + Vite	Builds rapides, typage fort, expérience de développement moderne
Style UI	Tailwind CSS + shadcn/ui	Système de design rapide et cohérent
Graphiques & visualisation	Recharts / Plotly.js	Bandes d'incertitude, distributions, histogrammes
Client API	Axios + TanStack Query	Récupération asynchrone, mise en cache, invalidation du cache
Framework BE	Python FastAPI	Asynchrone, documentation OpenAPI automatique, injection de dépendances
Framework ML	PyTorch	Standard pour les Transformers ; requis par Opacus
Confidentialité différentielle	Opacus (Meta)	Wrapper DP-SGD « drop-in » pour les boucles d'entraînement PyTorch
Imputeurs de référence	scikit-learn	KNNImputer, IterativeImputer (MICE), SimpleImputer
Métriques statistiques	scipy + scikit-learn	Wasserstein, MMD, AUC, ECE
Traitement des données	pandas + PyHealth	Chargement des jeux de données, prétraitement, masques d'absence de données
Authentification	JWT (python-jose) + bcrypt	Authentification sans état, hachage sécurisé des mots de passe
Stockage	SQLite + système de fichiers	Aucune surcharge d'infrastructure ; prêt pour PostgreSQL
Tests	pytest (BE) + Vitest (FE)	Tests unitaires, d'intégration et de composants
Environnement de dév.	uvicorn + npm run dev	Développement local à deux terminaux, sans Docker

16. Sources de données

Jeu de données	Taille	Accès	Rôle dans la plateforme
PhysioNet 2019 Sepsis Challenge	~40k patients 34 variables	Compte gratuit	Données d'entraînement principales ; fournit les étiquettes de sepsis pour l'évaluation de l'Axe 3
MIMIC-IV Clinical Database Demo	Sous-ensemble de 100 patients	Ouvert (ODbL)	Validation secondaire ; teste la généralisation au-delà de l'entraînement sur un seul jeu de données
Générateur synthétique	Illimité	Intégré	Modèle de copule gaussienne avec corrélations connues ; idéal pour les tests unitaires et la validation du pipeline

16.1 Interface DatasetLoader

Tous les jeux de données sont accessibles via une interface unifiée qui retourne une paire (X, mask) standardisée. Le remplacement des sources de données ne nécessite qu'une modification d'un seul fichier, sans refactorisation amont.

```
class DatasetLoader:
    def load(self) -> tuple[pd.DataFrame, pd.DataFrame]:
        # Retourne (X: valeurs d'analytes, mask: matrice binaire d'absence de données)
        ...
# Implémentations : PhysioNetLoader · MimicLoader · SyntheticLoader · HospitalLoader (futur)
```

17. Extensions futures

Extension	Priorité	Description
Architecture CVAE	**Élevée**	Ajouter un Auto-encodeur Variationnel Conditionnel comme second modèle d'imputation ; permettre la comparaison d'architectures sur les quatre axes d'évaluation
Modèle de diffusion léger	**Élevée**	Ajouter un modèle de diffusion basé sur le score pour l'imputation ; compléter l'étude comparative à trois architectures
Balayage complet ε DP-SGD	**Élevée**	Entraîner sur $\varepsilon \in \{1, 3, 10, \infty\}$; cartographier la frontière de Pareto qualité-confidentialité par architecture
Jeu de données MIMIC-IV complet	**Moyenne**	Étendre les résultats PhysioNet à MIMIC-IV accrédité ; valider la généralisation sur une population clinique plus riche
Intégration hospitalière réelle	**Moyenne**	Enregistrer un jeu de données hospitalier via DatasetLoader ; remplacement d'un seul fichier, aucun changement amont requis
Interface de matrice de permissions Admin	**Faible**	Grille à bascule éditable (ressource × action) dans le panneau Admin pour la personnalisation des rôles à l'exécution sans changement de code
API HL7 FHIR R4	**Faible**	Endpoint d'observation FHIR pour une intégration directe avec les DPI
Déploiement conteneurisé	**Faible**	Packaging Docker Compose ; migration de SQLite vers PostgreSQL pour la mise à l'échelle horizontale